

Question No:9

Problem Description:

You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet.

You are required to answer Q queries of type u, c , where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries.

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq u, v \leq N$$

- c is a lowercase alphabet
- s_i is a lowercase alphabet for all $1 \leq i \leq N$
- 1 is the root node

Input format

- First line: Two space-separated integers N and Q respectively
- Second line: A string s of length N (where the i^{th} character of s represents the character stored in node i)
- Next $N - 1$ line: Two space-separated integers u and v denoting an edge between node u and node v
- Next Q lines: An integer u and a space-separated character c

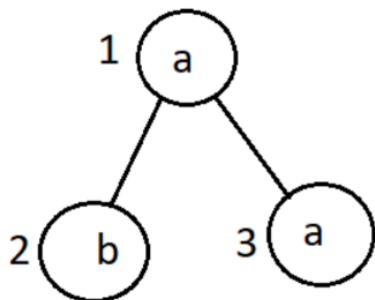
Output format

For each query, print the output in a new line.

Note: It is guaranteed that the input generates a valid tree.

Explanation for test case 1

Tree given in the sample input will look like that.



Number of nodes in the subtree of node 1 having 'a' stored in it is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 1
aba
1 2
1 3
1 a
```

EXPECTED OUTPUT

2

Test Case 2

INPUT (STDIN)

```
4 1
WSX
1 2
1 3
1 1
1 w
```

EXPECTED OUTPUT

10

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 100005
5
6  int head[MAXN];
7  int to[2 * MAXN];
8  int next[2 * MAXN];
9  int edgeCount = 0;
10
11 int parent[MAXN];
12 int order[MAXN];
13 int subtree[MAXN][26];
14
15 char s[MAXN];
16
17 void addEdge(int u, int v)
18 {
19     to[edgeCount] = v;
20     next[edgeCount] = head[u];
21     head[u] = edgeCount++;
22 }
23
24 int main()
25 {
26     int N, Q;
27
28     scanf("%d %d", &N, &Q);
29     scanf("%s", s + 1);
```

```
31 // Initialize adjacency list
32 for (int i = 1; i <= N; i++)
33     head[i] = -1;
34
35 // Read N-1 edges
36 for (int i = 0; i < N - 1; i++)
37 {
38     int u, v;
39     scanf("%d %d", &u, &v);
40
41     addEdge(u, v);
42     addEdge(v, u);
43 }
44
45
46 int top = 0;
47
48 order[top++] = 1;
49 parent[1] = 0;
50
51 for (int i = 0; i < top; i++)
52 {
53     int u = order[i];
54
55     for (int e = head[u]; e != -1; e = next[e])
56     {
57         int v = to[e];
```

```

58
59         if (v == parent[u])
60             continue;
61
62         parent[v] = u;
63         order[top++] = v;
64     }
65 }
66
67
68
69 for (int i = 1; i <= N; i++)
70 {
71     int c = s[i] - 'a';
72     subtree[i][c] = 1;
73 }
74
75 for (int i = N - 1; i > 0; i--)
76 {
77     int u = order[i];
78     int p = parent[u];
79
80     for (int c = 0; c < 26; c++)
81     {
82         subtree[p][c] += subtree[u][c];
83     }
84 }
85 for (int i = 0; i < Q; i++)
86 {
87     int u;
88     char c;
89
90     scanf("%d %c", &u, &c);
91
92     printf("%d\n", subtree[u][c - 'a']);
93 }
94
95 return 0;
96 }

```

Output:

Test Case-1:

```
3 1
aba
1 2
1 3
1 a
2
```

Test Case-2:

```
4 1
wsxX
1 2
1 3
1 1
1 w
2
```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236